CK-MPM: Paper-to-Code Guide

Vulkan sand implementation

This guide connects the current implementation to Michael Liu, Xinlei Wang and Minchen Li, *CK-MPM: A Compact-Kernel Material Point Method*, ACM TOG 44(4), 2025, doi:10.1145/3731155. Section and equation numbers below refer to that paper, not this guide. The local paper is `../../../CK-MPM Paper.pdf`.

The implementation follows the author's affine CUDA transfer and sand material. A paper reference identifies the corresponding idea; differences from the printed equations are stated explicitly.

Companion: *Mathematical Foundations and Derivations* starts with derivatives and Jacobians, derives Nanson's formula and the weak MPM force, then develops CK weights, APIC, MLS and sand plasticity. Source paths below are relative to the Vulkan project root.

1 Where to start

1. `shaders/interpolation.glsl`: compact kernel, grid offsets, weights and moments.
2. `shaders/g2p.comp`: fit affine velocity and update particles.
3. `shaders/p2g.comp`: transfer mass, affine momentum and stress.
4. `shaders/material.glsl`: sand stress and plastic correction.
5. `src/simulation.cpp`, `Simulation::advance()`: dispatch order, barriers and GPU timings.

2 Symbols and storage

Meaning	Notation	Code
Particle position/velocity	$\mathbf{x}_p, \mathbf{v}_p$	`position.xyz`, `velocity.xyz`
Grid spacing/timestep	$\Delta x, \Delta t$	`params.step.y`, `params.step.x`
Particle mass/volume	m_p, V_p	`params.step.z`, `params.step.w`
Grid mass	m_i	`nodes[i].w`
Grid momentum/velocity	$m_i\mathbf{v}_i, \mathbf{v}_i$	`nodes[i].xyz`; grid update changes momentum into velocity.
Stored elastic deformation	$\mathbf{F}_p$	`f0`, `f1`, `f2`: padded columns.

Meaning	Notation	Code
First Piola stress	$\mathbf{P}_p$	Evaluated in `sandMaterial()`; stored stress is $\mathbf{T} = \mathbf{P}\mathbf{F}^T$.
Affine velocity	$\mathbf{C}_p = \mathbf{B}_p\mathbf{D}_p^{-1}$	`c0`, `c1`, `c2`.
Moment matrices	$\mathbf{B}_p, \mathbf{D}_p$	Local `B`, `D` and `inverseMoment()`.
Dual-grid weight	$w_{i,p,k}$	`kernelWeight(o,w)`.
Plastic volume history	$\log J_p$	`plastic.x`, passed as `logJp`.

CPU layout: src/simulation_config.hpp, `Particle` and `Parameters`. Matching GPU layout and accessors: shaders/mpm_common.glsl. After plastic projection the stored F is an elastic state. The companion's elastic/plastic section distinguishes it from the full geometric deformation Jacobian used in Nanson's formula and density conversion.

3 Paper-to-code map

Paper	Implementation
§4.1, Eq. (4)	shaders/interpolation.glsl, `interpolation()`: the sine-corrected compact 1D kernel.
§4.2, Eq. (5)	`kernelWeight()` in the same file: product of the three axis weights.
§4.2, dual grids	`interpolation()`, `nodeOffset()` and `nodeIndex()` in shaders/mpm_common.glsl: shifts of $-\Delta x/4$ and $+\Delta x/4$ along each axis; eight nodes on each grid, sixteen total.
§4.3.1, Eq. (12)	shaders/p2g.comp: atomic accumulation of wm_p into grid mass on each grid.
§4.4.1, Eq. (20)	Same shader: affine contribution $m_p\mathbf{C}_p\mathbf{r}_{i,p,k}$ accompanies particle translation.
§4.4.1, D before Eq. (20)	`inverseMoment()` and shaders/g2p.comp: weighted offset outer products from both grids.
§4.4.2, Eq. (21)	G2P: weighted velocity–offset outer products form B; C is obtained by multiplying by the inverse moment.
§4.3.2, Eq. (14)	shaders/material.glsl, `sandMaterial()`: material stress after plastic correction; stores $\mathbf{P}\mathbf{F}^T$. Detailed return mapping comes from author source.
§4.3.2; §4.5	shaders/p2g.comp, `contribution`: author's moment-based stress transfer, not a literal kernel-gradient evaluation of Eq. (15).
§4.3.3, Eq. (16)	shaders/grid_update.comp: normalize momentum by mass, add gravity, apply sticky boundaries independently to each grid. Stress momentum has already entered during P2G.
§4.3.4	G2P averages weighted velocities from both grids; the printed Eq. (17) distinction is explained below.
§3.2; §4.3.5, Eq. (19)	G2P: deformation update using fitted C and advection using updated particle velocity.

Paper	Implementation
§6.2, Sand Armadillos	`sandMaterial()`, `shaders/material_update.comp`, and `loadParticles()` in `src/simulation_config.cpp`: sand plasticity, armadillo assets and opposing velocities.
§5, implementation	`src/simulation.cpp`: separate Vulkan dispatches, barriers and timing queries. Storage and scheduling differ from the fused sparse CUDA implementation.

4 Kernel and affine transfer equations

For a cell-local axis coordinate d on a shifted grid,

$$w_0(d) = 1 - d + \frac{\sin(2\pi d)}{2\pi}, \qquad w_1(d) = 1 - w_0(d),$$
$$w_{xyz} = w_x w_y w_z.$$

This evaluates paper Eq. (4) within the occupied cell and uses the partition-of-unity identity in Eq. (8). The quadratic branch of `interpolation()` supplies the alternative B-spline kernel for comparison.

Let $k \in \{-1, +1\}$ index the two grids, let $\mathbf{r}_{i,p,k} = \mathbf{x}_{i,k} - \mathbf{x}_p$ be a physical offset, and let $\mathbf{u}_{i,k}$ denote updated grid velocity. Then

$$\mathbf{D}_p = \frac{1}{2} \sum_k \sum_i w_{i,p,k}\, \mathbf{r}_{i,p,k} \mathbf{r}_{i,p,k}^T,$$
$$\mathbf{B}_p = \frac{1}{2} \sum_k \sum_i w_{i,p,k}\, \mathbf{u}_{i,k} \mathbf{r}_{i,p,k}^T,$$
$$\mathbf{C}_p = \mathbf{B}_p \mathbf{D}_p^{-1}.$$

G2P omits both factors of $1/2$ because they cancel in the fit. Its offsets are measured in cell units, so the code computes

$$\mathbf{C}_p = \frac{1}{\Delta x} \widehat{\mathbf{B}}_p \widehat{\mathbf{D}}_p^{-1}.$$

`inverseMoment()` returns the inverse of the averaged D in physical units, evaluated at the current particle position for P2G.

The P2G shader combines affine momentum and stress:

$$\mathbf{T}_p = \mathbf{P}_p \mathbf{F}_p^T,$$
$$\mathbf{A}_p = m_p \mathbf{C}_p - \Delta t\, V_p \mathbf{D}_p^{-1} \mathbf{T}_p,$$
$$\mathbf{q}_{i,k} \mathrel{+}= w_{i,p,k}(m_p \mathbf{v}_p + \mathbf{A}_p \mathbf{r}_{i,p,k}),$$
$$m_{i,k} \mathrel{+}= w_{i,p,k} m_p.$$

Here $\mathbf{q}$ is grid momentum. In particular, D inverse multiplies T on the *left*, matching `contribution` in the author's `../CK-MPM/include/mpm_algorithm.cuh`, `G2P2G()`. The companion's section "MLS reconstruction and a convention audit" derives the conventional column-vector force ordering $\mathbf{TD}^{-1}\mathbf{r}$. That order differs from the source ordering shown above in general. A shared code/reference result does not resolve this mathematical difference; keep it as an explicit audit point rather than asserting equivalence.

The MLS shape functions in paper Eqs. (22)–(25) explain the moment-based approach; our code uses the author's concrete formulation rather than building a general polynomial basis.

Particle velocity and deformation use

$$\mathbf{v}_p^{n+1} = \frac{1}{2}\sum_k \sum_i w_{i,p,k}\mathbf{u}_{i,k},$$
$$\mathbf{F}_p^{\text{trial}} = (\mathbf{I} + \Delta t\, \mathbf{C}_p)\mathbf{F}_p^n,$$
$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t\, \mathbf{v}_p^{n+1}.$$

The velocity average follows the author's CUDA loop. **Printed paper Eq. (17) includes grid-mass/particle-mass factors that this shader does not use.** The mapping to §4.3.4 is conceptual, not a claim of literal equality to that printed equation. Fitted C supplies the deformation update instead of explicitly summing the kernel derivatives in Eq. (18).

5 Sand model and source provenance

The paper identifies Drucker–Prager sand and its example parameters in §6.2. The material overload of `ComputeStress()` in `../CK-MPM/include/mpm_material.cuh` uses the model `kDruckerPragerStvkhencky`. Its callers are in `../CK-MPM/include/mpm_algorithm.cuh`.

In `shaders/material.glsl`, `svd3()` factors F using one-sided Jacobi sweeps. `sandMaterial()` evaluates logarithmic stretches, projects plastic strain, updates $\log J_p$, reconstructs F and evaluates stress. `shaders/material_update.comp` stores corrected deformation, stress and plastic history. Friction angle is configurable; the corresponding author CUDA overload fixes its yield coefficient at 30°.

The paper's example uses 511,902 particles, $E = 10^4$, $\nu = 0.4$, friction angle 30° and gravity -2. JSON settings can differ. `loadParticles()` uses the author assets with initial velocities $(0, 0, -0.5)$ and $(0, 0, 0.5)$; `parameters()` rescales mass/volume when sampling fewer particles.

6 Execution and implementation choices

`Simulation::advance()` dispatches each substep in this order:

$$\text{clear} \longrightarrow \text{P2G} \longrightarrow \text{grid update} \longrightarrow \text{G2P} \longrightarrow \text{material/plastic update.}$$

Corrected stress at the end of a substep feeds the next P2G. Initial F is identity and stored stress is zero. Advection and the elastic deformation update occur inside G2P.

Unlike the fused G2P2G and blocked sparse AoSoA storage in §5.1, this code uses separate passes, a dense grid, padded particle structs and direct global atomics. These differences matter when comparing performance.

Automatic acoustic stepping is in `SimulationConfig::timestep()`. Offline scheduling also limits particle travel. Manual live stepping in `live()` uses positive dt and `steps_per_frame` directly; these are application controls, not stability or real-time guarantees from the paper.

7 Tests and application infrastructure

Paper §6.1 studies full momentum-conservation experiments, and §6.2 compares kernels. Our `tools/evaluate.py` checks small NumPy stage fixtures, analytic affine fields, two-step state reuse and GPU safety reductions. `tools/taichi_reference.py` is an affine stage reference,

while tools/cuda_material_reference.cu executes the author's material/SVD. These checks do not reproduce the full paper experiments.

Source	Application role
shaders/particle_health.comp, Simulation::health()	GPU finite-state, domain/stencil and maximum-speed reduction.
src/renderer.cpp, shaders/particle.vert, shaders/particle.frag	Particle spheres, camera rendering and direct reads of live GPU particle state.
src/vulkan_context.cpp	Vulkan memory, command submission, presentation and validation.
src/scene_cache.cpp, calculate()/play() in src/main.cpp	Save sampled particle positions and replay them.